Stat 415 Lab 4 - Predictive Modeling

Chan Lee

May 24, 2025

1 Introduction

Our goal is to develop various predictive models using housing price data from Illinois and Wisconsin. We foresee a plethora of other applications that our work can have. For example, our analysis can help those considering buying a house in these two states to make a more informed decision.

We also seek to evaluate their performance through different statistical analyses and visualization techniques, including simple neural networks, decision trees, linear regression, and many more. Hopefully, this can also provide insight into the pros and cons of each of these models.

2 Exploratory Data Analysis

We perform analyses on the following two datasets, obtained from realtor.com:

1. **IL_housing_price.csv**: contains housing data on real estate sold in Illinois between January 1st, 2017 and April 1st, 2024.
2. **WI_housing_price.csv**: contains housing data for 50 foreclosure properties (which are properties being sold after the previous homeowner's right to the property is terminated, usually due to default) sold in Wisconsin during the same period. Foreclosure properties are usually sold at public auctions, where they pay off the mortgage debt through the proceeds.

Both datsets include the variables in the following table:

Variable	Description
property_url	URL of the property listing on realtor.com
style	Style of the house (e.g., condo, single house, mobile home)
beds	Number of bedrooms
full_baths	Number of full bathrooms
half_baths	Number of half bathrooms
sqft	Total square footage of the house
year_built	Year when the house was built
sold_price	Sale price of the house in USD (target variable)
lot_sqft	Square footage of the lot
latitude	Latitude coordinate of the property
longitude	Longitude coordinate of the property
county	County where the property is located
stories	Number of stories in the house
parking_garage	Number of parking garages

3 Predicting Illinois Housing Prices

The first part of our analysis focuses on predicting the prices of houses in Illinois using *IL_housing_price.csv*. We use various methods to do this, including linear regression, linear regression with L1 and L2 regularization, decision trees, random forests, and neural networks.

3.1 Using Linear Regression

We trained a linear regression model on 80 percent of the Illinois dataset, and reserved the other 20 percent for testing. *Sold_prices* was the target variable, and all the others (except *latitude*, *longitude*, and *property_url*) were used as covariates. We did one hot encoding on the two categorical variables: *style* and *county*. Note that we remove one of the houses from the test dataset since it is extremely massive (it has almost 8,000 times the median number of square feet of houses in the test dataset, and thus, has a predicted price that is significantly higher than that of the other houses. In other words, we removed the one outlier to visualize the distributions more elegantly.

Figure 1 compares the distribution of true prices and the prices we predict using our trained model, of sold houses in Illinois. We can see here that the distribution of true prices is heavily right-skewed with a long tail, typical of housing markets where most homes are moderately priced and a few are very expensive. The distribution of the predicted prices is similar, but is less right-skewed and more bell-shaped. This alignment suggests the linear model captures Illinois house prices fairly well, yet the right-skewedness suggests that a log transformation of sold prices would make a better response for linear regression. Furthermore, we are concerned about the model's ability to accurately predict high-priced homes, since there are so few of them in our training set and since it fails to model the diminishing returns on the size of homes.

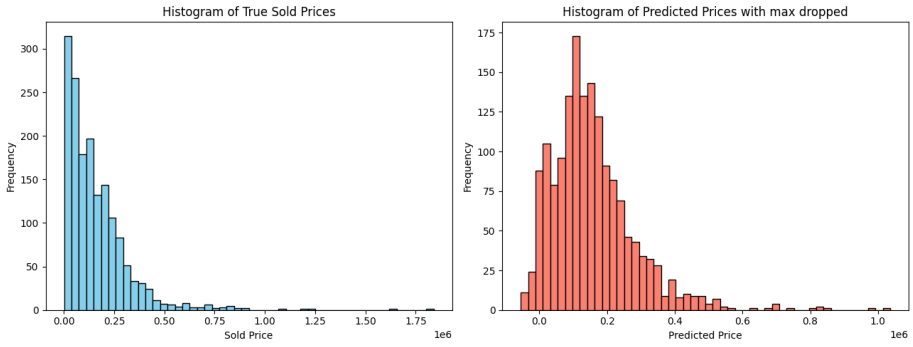


Figure 1: Comparing Distributions of True vs Predicted House Prices in Illinois Dataset (Test Split)

We also visualize the distributions of the errors (after removing the outlier) in our predictions in Figure 2. One can see that the errors are normally distributed, which aligns with the normality of errors assumption of linear models. Moreover, they are centered around zero, meaning that the predictions are generally unbiased. There is, however, a mild right skew and long tails, suggesting that there are still some larger mistakes, particularly in overestimation. This shape is typical for housing price data, where a linear model struggles to accurately model expensive properties.

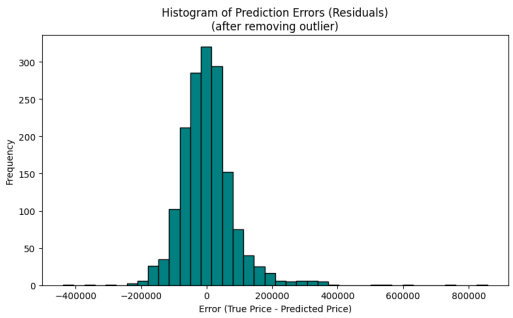


Figure 2: Distribution of Linear Model Errors

3.2 Using Lasso and Ridge Regression

In addition to our linear model, we do lasso regression and ridge regression on the same training and testing split as before. For each of these three models, we record the Mean Squared Error (MSE) during inference in Table 1. We see that Lasso does slightly better than Linear Regression, and Ridge does slightly better than Lasso. This suggests that adding L2 regularization helps reduce overfitting and stabilizes the model by penalizing large coefficients, especially in the presence of multicollinearity or one-hot encoded categorical variables. Lasso also improves performance marginally by shrinking some coefficients to zero, but not as effectively in this case—likely because no dominant irrelevant features exist to discard entirely. Overall, regularization yields modest but measurable improvement in predictive accuracy.

Model	**MSE (Mean Squared Error)**
Linear Regression	2.2037×10^{14}
Lasso Regression	2.2033×10^{14}
Ridge Regression	2.1998×10^{14}

Table 1: Comparison of MSE for Linear, Lasso, and Ridge Regression Models

3.3 Using Decision Trees

We then trained a decision tree model using the same dataset and training/testing split. We get the MSE and RMSE values, during inference, which are displayed in Table 2. Note that the MSE value is much lower than that of the previous three models.

Metric	**Value**
MSE	1.45×10^{10}
RMSE	1.20×10^{5}

Table 2: MSE and RMSE for the Decision Tree Regressor

Note that we can visualize our decision tree. Because the tree contains an extremely high number of nodes, we decided to only show a small section of this decision tree, as seen in Figure 3. Note that the node at the lowest level in this figure denotes houses with more than 33.5 stories, and the decision attribute at the next level is the size of the houses in sqft. In this figure, we are only shown the node denoting houses with sqft greater than 835.5, but the second arrow point out from the node at the lowest level shows that we also have a node for houses with sqft less than 835.5. Subsequently, we can see that the training algorithm continues to choose a new decision attribute at every level and divides up the dataset according to that attribute.

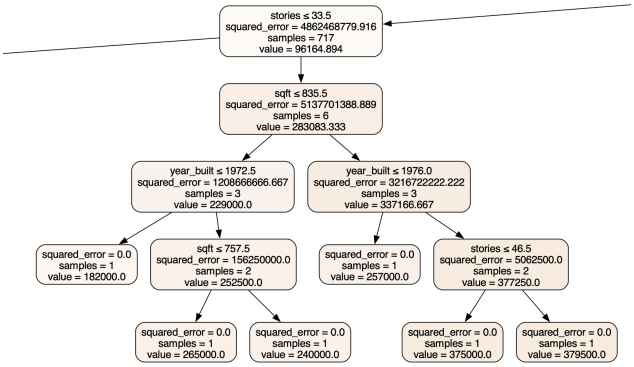


Figure 3: A Section of the Decision Tree

3.4 Using Random Forests

We then trained a random forest model using the same dataset and training/testing split. We get the MSE and RMSE values written in Table 3. This improves from the decision tree model, which makes sense since random forests decrease the variance.

Metric	Value
MSE	7.47×10^9
RMSE	8.64×10^4

Table 3: MSE and RMSE for the Random Forest Regressor

As seen in Figure 4, the random forest predicts the true sold prices fairly well. We notice that it does so better than the linear model does, since it has more of a right skew and has a longer right tail compared to the linear model, which was more bell-shaped. However, random forest predictions still have limitations: it fails to predict the prices of very expensive homes (namely, those that cost at least 1.4 million dollars). This is clear in the truncated right tail of the random forest predictions. This is likely due to the ensemble-averaging nature of random forests, which tend to bias predictions towards the center of the study. the data distribution.

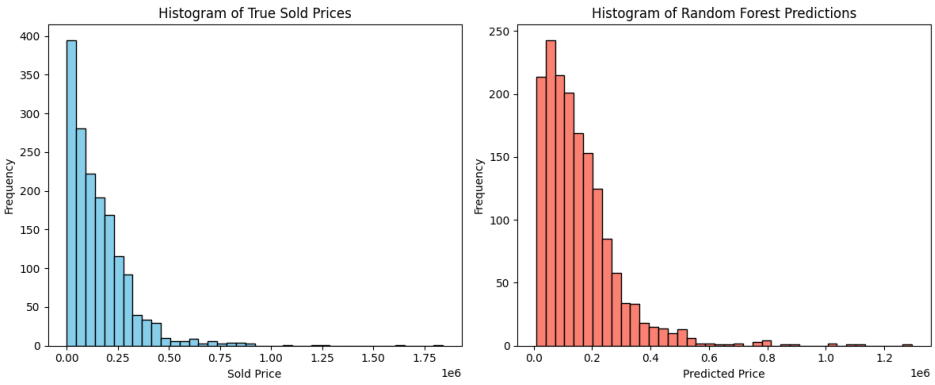


Figure 4: Histograms of Sold Prices: True vs Random Forest Predictions

3.5 Using Neural Networks

We then trained a 3 layer neural network with sigmoid activations using PyTorch. We trained for 100 epochs, used the MSE loss function, and the Adam optimizer. We used the same datasets as in previous sections and the same training/testing split. We get the MSE and RMSE values displayed in Table 4. It did worse than random forests, but still outperformed decision trees and Linear, Lasso, and Ridge regression (refer to Tables 1 2, and 3). This shows that the neural network is a relatively effective and efficient method for house price prediction.

Metric	Value
MSE	1.71×10^{10}
RMSE	1.31×10^{5}

Table 4: MSE and RMSE for the NN: 3 Layers, Adam Optimizer, Sigmoid Activation

We can see in Figure 5 that the neural network pretty accurately predicts the sold prices of the test dataset. We see that it fails to capture the diversity of prices for cheaper houses, as it mostly predicts homes to be 50,000 dollars. Hence, the histogram on the right is much more right skewed, failing to capture the gradual decline in frequency as sold price increases.

This makes sense, since neural networks are known to converge to the mean. In our case, this appears to be happening: the model's predictions are tightly clustered around the average housing price, with a very low standard deviation. We confirmed this by comparing the standard deviation of predictions to that of the true target values. To test if this is a consistent issue, we retrained the model from scratch using multiple random seeds. Across runs, we observed that the network frequently collapsed to the mean, indicating that the problem is structural—likely due to limited number of epochs, sigmoid activations, or some other issue. This confirms the known risk of mean convergence in underpowered neural networks.

Although the distribution of the neural network predictions deviate from the true sold prices in this manner, we hypothesize that by increasing the number of epochs, we can increase the accuracy of our predictions. Furthermore, it is possible that the sigmoid activations at each layer limited expressiveness. By removing this or substituting it with something else, we could potentially improve performance. All in all, despite only having three activation layers and 100 epochs, we find that its performance (with respect to MSE) is comparable to that of decision trees, making it an efficient and effective model.

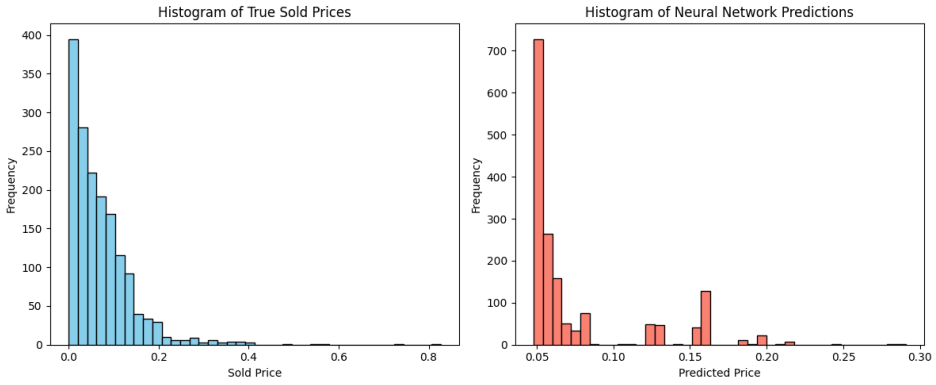


Figure 5: Histograms of Sold Prices: True vs Neural Network Predictions

3.5.1 Ablation Study

We performed an ablation study by changing the number of activation layers to 5, the activation function to ReLU, and the optimizer to SGD. We did not use any dropout or weight decay. We find that the MSE value, shown in Table 5, increased by changing these variables

Metric	Value
MSE	4.47×10^{10}
RMSE	2.11×10^{5}

Table 5: MSE and RMSE for the NN: 5 Layers, SGD Optimizer, ReLU Activation

The increase in MSE from 1.71×10^{10} to 4.47×10^{10} could have occurred because unlike Adam, which adaptively adjusts learning rates and typically converges faster and more stably, SGD requires careful tuning of the learning rate and benefits from momentum to perform well—without these, training can stagnate or diverge. Also, it is possible that adding more layers significantly increases model complexity and the number of parameters, making the network harder to train and more prone to issues like vanishing gradients or poor generalization if not paired with techniques like input normalization, dropout, or learning rate scheduling. Lastly, while ReLU activations are generally more effective than sigmoid in deeper architectures, they are sensitive to input scale and can lead to inactive neurons if not properly initialized or normalized. Overall, the performance drop is likely due to the combined effects of insufficient optimization and over-complexity introduced without compensatory training adjustments.

4 Predicting Illinois Housing Prices by *County*

Our next goal is to use visualize our predictions for house prices in Illinois based on county. We see in Figure 6 a map of Illinois counties, colored by the average sale price of foreclosure properties.

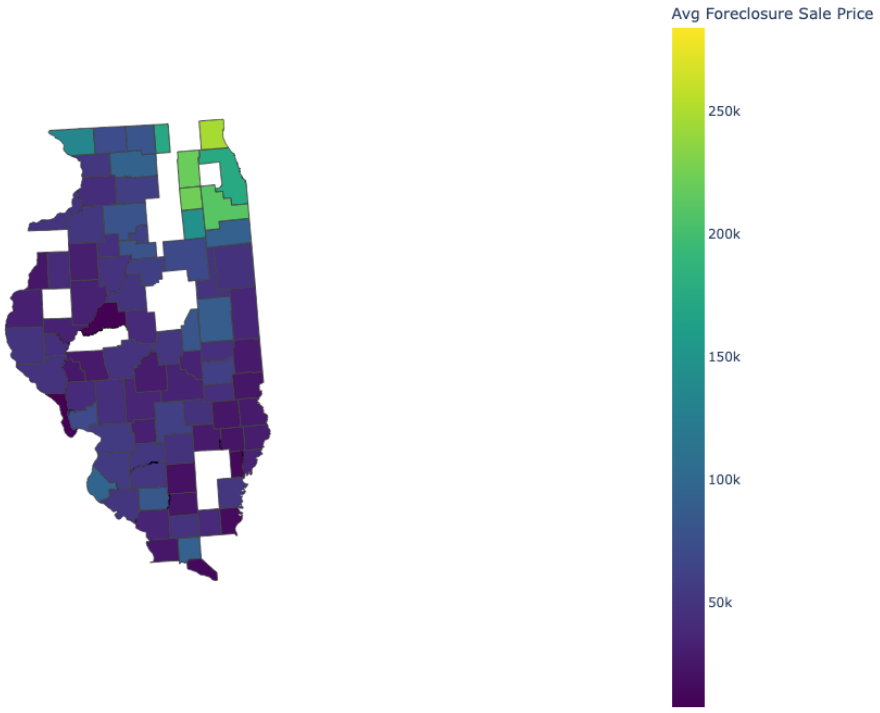


Figure 6: Map of Illinois Counties by Avg. Foreclosure Sale Price

We then display the predictive errors by county using the random forests model we implemented previously, in Figure 7. We first notice that the areas where the average foreclosure sale prices are the highest (i.e. in the Northeastern part of Illinois) has relatively high error. This makes since all of our models were fairly bad at predicting prices of the most expensive homes. Another glaring observation is that some of the southern, more rural counties have the highest average absolute error by a huge margin. This may be due to the lack of samples from these areas due to the sparsity of human settlement in rural lands. This makes it more likely to generalize poorly to said areas.

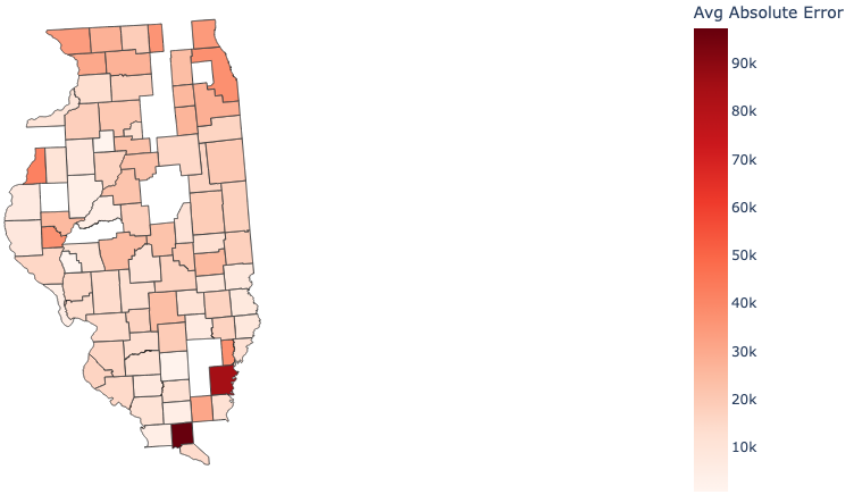


Figure 7: Map of Illinois Counties by Predictive Error

5 Predicting Wisconin Housing Prices

We use three models: linear regression, a random forest, and a neural network with three layers. The neural network uses the Adam optimizer and sigmoid activation function, and is trained over 100 epochs. We see the MSE of all three models in Table 6. The neural network does the best at predicting, followed by the random forest and finally the linear regression model. This can also also be visualized in Figure 8. Here, we see that the neural network and random forest predict the true sold prices of the test datset most accurately, while the linear regression predicts more uniformly across roughly evenly spaced out prices.

Compared to the same models used to predict Illinois housing prices, we notice that these models achieve significantly lower MSE values. Several factors likely contribute to the lower MSE on Wisconsin data. Wisconsin's sale prices may simply cluster around a lower average with less extreme variation than Illinois, so even similar absolute errors square to smaller values. If Wisconsin's market has fewer ultra-high-end or outlier transactions, the worst errors—and thus overall MSE—are reduced. A more homogeneous housing stock and cleaner, more complete feature data (beds, baths, square footage, etc.) can also strengthen the relationship between predictors and sale price, making models easier to fit accurately compared to the more diverse, high-variance Illinois market.

Model	**MSE**	**RMSE**
Neural Network (3 Layers, Adam, Sigmoid)	5.28×10^9	7.27×10^4
Linear Regression	1.62×10^{10}	1.27×10^5
Random Forest	5.80×10^9	7.62×10^4

Table 6: Comparison of MSE and RMSE for Models Trained on Wisconsin Housing Dataset

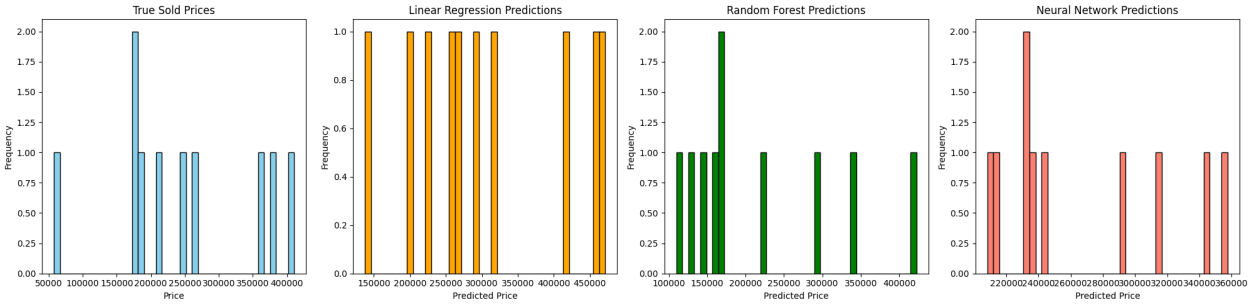


Figure 8: Distributions of Real and Predicted Wisconsin Housing Prices

6 Transfer Learning

6.1 Part 1

Step by step, we: (1) fit a linear regressor on Illinois data; (2) used those learned coefficients to predict Wisconsin prices; (3) computed error metrics on the Wisconsin test set; and (4) compared these to Wisconsin-specific baselines. The poorer transfer performance likely stems from distributional shifts in sale-price scales, feature distributions (e.g. typical lot sizes, year-built ranges), and region-specific market drivers. Without retraining or domain adaptation, the Illinois-trained linear model cannot fully capture Wisconsin's different housing dynamics.

When we take the linear model trained on Illinois and apply it directly to Wisconsin, we observe a transfer MSE of 2.00×10^{10} (RMSE 1.41×10^{5}). By contrast, a linear regression trained *on* Wisconsin achieves an MSE of 1.62×10^{10} (RMSE 1.27×10^{5}), and our more flexible models (random forest, neural net) on Wisconsin attain even lower errors ($\sim$ 5–6 $\times 10^{9}$). See Table 7. Thus, the transferred Illinois model performs noticeably worse than a model tuned to Wisconsin, indicating that naive weight-transfer across these two regional markets degrades predictive accuracy.

Metric	**Value**
Transfer MSE on WI data	2.00×10^{10}
Transfer RMSE on WI data	1.41×10^{5}

Table 7: Performance of Illinois-trained linear model when transferred to Wisconsin data

6.2 Part 2

Compared to the models trained directly on the Wisconsin housing dataset, the fine-tuned neural network achieves the best overall performance. With a transfer MSE of 4.16×10^{9} and RMSE of 6.45×10^{4}, it outperforms the standalone Wisconsin-trained neural network (MSE 5.28×10^{9}), random forest (MSE 5.80×10^{9}), and linear regression (MSE 1.62×10^{10}). This indicates that initializing the neural network with weights trained on the Illinois data provided a more effective starting point than random initialization, allowing the model to better capture price dynamics in the smaller Wisconsin dataset. Refer to Table 8 for the results.

These results suggest that transfer learning is beneficial in this setting. Pretraining on the larger Illinois dataset helped the model learn general patterns about housing structure and price relationships that transferred well to a different, but related, geographic market. Fine-tuning on Wisconsin for only a few epochs allowed the model to adapt to local variations without overfitting, achieving superior accuracy. This shows that even a simple form of transfer learning can offer significant improvements when target data is limited.

Metric	**Value**
Transfer MSE after Fine-Tuning	4.16×10^{9}
Transfer RMSE after Fine-Tuning	6.45×10^{4}

Table 8: Performance of Fine-Tuned Neural Network on Wisconsin Data After Transfer Learning

7 Conclusion

In this lab, we explored multiple predictive modeling techniques to estimate housing prices in Illinois and Wisconsin. Our analysis demonstrated the strengths and limitations of various models, including linear regression, decision trees, random forests, and neural networks. We found that tree-based models, particularly random forests, consistently outperformed linear models in both states due to their ability to capture nonlinearities and interactions in the data. Neural networks offered competitive performance and showed promise as a lightweight yet expressive alternative, especially when properly tuned.

We also investigated the potential of transfer learning by pretraining a neural network on the Illinois dataset and fine-tuning it on the smaller Wisconsin dataset. This approach led to the best overall performance on Wisconsin, highlighting the value of leveraging knowledge from related but larger datasets. Our findings underscore the importance of model selection, regularization, and training strategy when working with real-world housing data and support the practical utility of transfer learning in low-data scenarios.